COMP 4959: Lab 1

Submit a zip file containing your two source files, `sort.rs` and `primes.rs`. Maximum score: 19

1. Implement a program in Rust to sort records read either from standard input or from a file. We'll call our program `sort`. When `sort` is invoked on the command-line

   - without any argument, it reads from standard input;
   - with one or more arguments, then the argument after the program name is the file to read; the remaining arguments are simply ignored.

   In the latter case, print an error message and exit if the program fails to open the specified file.

   Use the following type for records:

   ```
   struct Record {
     first_name: String,  // can be any word
     last_name: String,   // can be any word
     score: u8,           // must be between 0 and 100 inclusive
   }
   ```

   A record contains a first name, a last name and a score. The first and last names can be any word, but the score must be between 0 and 100 inclusive. An input line may contain information for one record or may be invalid. The line can be invalid either because the information is incomplete (has fewer than 3 words) or the score is invalid. Invalid input lines are simply skipped.

   The following shows 5 sample input lines:

   ```
   homer  simpson  25  # bad at nuclear engineering
    ned flander  12abc  !! invalid score
      waylon   smithers  100  ! high score
    homer simpson   25  @ duplicate
     gary  chalmers  5
monty  burns
   ```

   Note that the second and last lines above are invalid.

   For an input line, the first word (if any) specifies the first name, the second word (if any) specifies the last name, and the third word (if any) specifies the score. Additional words are simply ignored.

   Store the valid read records in a vector. After finishing reading records (on end-of-file), the program proceeds to sort the vector of records and then display the records to standard output. The sorted output is in descending order of scores, and if multiple records have the same score, they are then in ascending order of last names. Finally, records with the score and last name are in ascending order of their first names.

   The following shows the output for the above sample input lines:

   ```
   100 smithers, waylon
    25 simpson, homer
     5 chalmers, gary
   ```

   Note carefully the output format. Note also that duplicates are not shown. Put your code in a file named `sort.rs`.

2. Write a Rust program to find the size of the largest set of 6-digit primes that are permutations of one another.

   Your implementation must have a function

   ```
   // precondition: m < n
   fn primes(m: usize, n: usize) -> Vec<usize>
   ```

   that returns a vector containing all primes p in ascending order satisfying $m \leq p < n$. Base your implementation on the Sieve of Eratosthenes. For testing, your program must print out the size of the set of all 6-digit primes as well as the size of the largest set of 6-digit primes that are permutations of one another. Name your file `primes.rs`.